

Veloce

Post-Quantum Cryptography SDK

V1 Engineering Plan

Organization	Lightrider Inc
Product	Veloce: cryptographic search SDK + FIPS entropy generator
Version	0.2 (implementation-aligned review)
Date	August 10, 2026
Prepared for	Martin Guo, Quantum Applied Scientist
Status	Linux reference implementation complete; Windows/macOS work planned

Abstract. Veloce V1 delivers two products in one SDK: **qSearch**, a high-performance cryptographic discovery tool (source, dependencies, certificates, network) that produces a Cryptography Bill of Materials (CBOM); and the **Veloce crypto core**, a locally deployed entropy generator and PQC service built on the wolfCrypt FIPS 140-3 module (certificate #4718) seeded exclusively by wolfSSL's ESV-certified wolfEntropy source, exposing ML-KEM, ML-DSA, and hybrid TLS 1.3 through a Python SDK. Cloud connectivity to the Lightrider EMS is optional, off by default, and includes a user-controllable cloud-entropy mix-in that never weakens the local FIPS entropy claim. The checked-in implementation is currently Linux x86-64; native Windows and macOS deliverables remain separate platform tracks in this same repository.

Contents

1	Executive Summary	2
2	Scope and Requirements	4
2.1	In scope (V1)	4
2.2	Out of scope (V1)	4
2.3	Standards and policy baseline	4
3	System Architecture	4
3.1	Component responsibilities	5
4	Pillar A: qSearch, Cryptographic Search	5
4.1	Collector adapters (replaceable)	6
4.2	Lightrider-owned layers	6
4.3	One internal finding format	6
4.4	Detection targets and outputs	6
4.5	Coverage assurance and outlier names	7
4.6	Optional AI triage	7
4.7	Performance approach	7
5	Pillar B: Entropy Generator and FIPS Crypto Core	7
5.1	Entropy pipeline	8
5.2	FIPS module integration	8
5.3	PQC provider outside the FIPS boundary	9
5.4	Hybrid TLS 1.3	9
5.5	Performance plan	9
6	Pillar C: EMS Cloud Integration	9
6.1	Modes and configuration	10
6.2	User-controllable cloud entropy (V1 addition)	10
7	APIs and Deliverables	10
7.1	Public Python API (17 functions, fixed by the guidance)	10
7.2	Deliverables checklist	11
7.3	Platform-separated development and release layout	11
7.4	Startup branding	11
8	Security and Compliance	12
8.1	Licensing and distribution model	12
9	Test Gates and Acceptance	13
10	Execution Guide: What To Do, In Order	13
10.1	Component-to-repository traceability	14
11	Risks and Open Items	14
A	Build Recipes	15
B	Manual Two-Host Interoperability Test	16
C	CBOM Field Mapping (SDK → workbook)	16
D	References	16

1 Executive Summary

Veloce is Lightrider’s V1 PQC SDK. It comprises a cryptographic discovery component (qSearch) and a FIPS 140-3 crypto core with a validated entropy pipeline. Both operate fully offline; an optional EMS connection provides fleet telemetry and user-controlled cloud entropy.

Document organization. This document is the implementation specification for Veloce V1 and is sufficient for reimplementing on its own. Sections 3–6 define the components, Section 7 the public API and deliverables, Section 8 the security controls, Section 9 the acceptance tests, Section 10 the build order with a component-to-repository map, and the appendices the exact build commands.

Implementation snapshot (2026-08-10). The checked-in Linux implementation contains the C++ agent, pure-Python client, Rust CLI, native Rust qSearch source/system scanner, CBOM outputs, and 33-test gate battery. The current source scanner is a single-threaded recursive literal-pattern engine, not yet the parallel collector design described as the target below. Hybrid-TLS policy selection exists, but the TLS data plane, native Windows/macOS transports, native packages, dependency/PCAP collectors, and EMS network client are not implemented. This distinction is normative: target requirements must not be reported as delivered functionality.

The two pillars

Pillar A: qSearch (cryptographic search).

A Rust-based discovery engine with a native literal-pattern and PEM-certificate collector today, with replaceable dependency and network collectors planned. It finds RSA, DH, ECDH, ECDSA and other quantum-vulnerable indicators in source code and certificates, and emits JSON, CSV, CycloneDX CBOM, and OMB M-23-02 inventory fields. Lightrider owns the normalization, correlation, classification, CBOM generation, risk scoring, and reporting layers.

Pillar B: Entropy generator and FIPS crypto core.

A native C++17 agent embedding the wolfCrypt FIPS 140-3 validated module (CMVP cert #4718) whose SP 800-90A Hash_DRBG is seeded only by wolfSSL’s ESV-certified wolfEntropy source (SP 800-90B health tests, fail-closed). The agent provides ML-KEM-768 key establishment and ML-DSA signatures, consumed from a pure-Python SDK over an authenticated local IPC channel. Hybrid TLS 1.3 policy selection is present; the sample handshake/data plane remains a G3 deliverable.

Technical stack

The product is designated **Veloce**; its components are qSearch, the Veloce Agent, the Veloce Python SDK, the Veloce CLI, and EMS (`light-rider-platform`).

Layer	Stack element	Technical basis
Crypto module	wolfCrypt FIPS 140-3 module v5.2.1, CMVP cert #4718, dynamically loaded	The guidance mandates a shared library/DLL (no static linking) and the purchased bundle carries this exact validated module; autotools (<code>--enable-fips=v5</code>) is its only FIPS-capable build path.
Entropy source	wolfEntropy (ESV), <code>--enable-wolfEntropy=nofallback</code> , wired via <code>wc_SetSeed_Cb()</code>	The FIPS module itself makes no entropy claim; SP 800-90B compliance therefore requires an external ESV-validated source, which the bundle provides with built-in RCT/APT health tests. V1 entropy is software-only; hardware QRNG (IDQ Quantis) and the Quantropi quantum-entropy service are post-V1 integrations.
PQC provider	ML-KEM / ML-DSA compiled <i>beside</i> the FIPS boundary	The v5 FIPS build strips SHAKE/XOF from in-boundary SHA-3, so FIPS 203/204 code cannot link inside it; reported as <code>pqc_inside_fips_boundary: false</code> until wolfSSL’s FIPS v7 “PQ-FS” module ships (Section 5.3).

Agent	C++17 native service	The Windows instruction set specifies C++17; wolfCrypt is a C library, making C++ the lowest-friction FFI; all keys and FIPS state must reside in one local process.
qSearch	Rust	The current standard-library-only implementation recursively scans UTF-8 files with literal patterns and inspects PEM OIDs; the target engine adds a compiled multi-pattern matcher and parallel walking. It links no wolfCrypt code.
SDK client	Pure Python over local IPC	The guidance mandates a pure-Python IPC client that never loads the crypto library; private keys are represented only as opaque handles.
Cloud entropy	EMS-delivered packets as DRBG <i>additional input</i> , default off	SP 800-90A permits additional input with zero credited entropy, so cloud entropy can strengthen output without altering the FIPS/ESV claims; the source documents prohibit entropy leaving the device, and this design sends none.
Platforms	Linux x86-64 implemented; Windows 11 x86-64 next; macOS separate track	Platform adapters and release artifacts remain isolated under one repository; no unimplemented platform may be advertised as supported.
Licensing	Lightrider commercial EULA wrapping the wolfSSL commercial license	The agreement (v.01-24) permits object-code redistribution of the Combined Program with notices retained (Section 8.1).

Terminology

Term	Meaning in this document
PQC	Post-quantum cryptography: algorithms that resist attack by quantum computers.
Quantum-vulnerable	Cryptography a large quantum computer could break: RSA, DH, ECDH, ECDSA.
ML-KEM / ML-DSA	The NIST-standardized post-quantum key-establishment (FIPS 203) and signature (FIPS 204) algorithms.
Hybrid TLS	A TLS 1.3 handshake combining a classical and a post-quantum key exchange; secure if either component holds.
FIPS 140-3	The U.S. government standard for validating cryptographic modules; validation is issued as a numbered CMVP certificate.
Module boundary	The exact code covered by a FIPS certificate; anything outside it carries no validation claim.
OE	Operational environment: the specific operating system and hardware a validation was tested on.
Entropy	Unpredictable data used to seed random-number generation; the root of all key security.
ESV	NIST Entropy Source Validation: certifies an entropy source under SP 800-90B.
DRBG	Deterministic random bit generator: expands a small entropy seed into unlimited random output (SP 800-90A).
RCT / APT	The two continuous entropy health tests (repetition count, adaptive proportion) required by SP 800-90B.
KAT / CAST	Known-answer test / conditional algorithm self-test: fixed-input checks a FIPS module runs on its own algorithms.
CBOM / SBOM	Cryptography / software bill of materials: an inventory of where and how cryptography (or software) is used.
Collector	A replaceable tool qSearch uses to gather raw evidence (source scan, certificate scan, network capture).
Correlate	Merge findings from different collectors that refer to the same asset.
Remediation	Changing or replacing vulnerable cryptography; V1 discovers and reports only.
IPC	Inter-process communication: the authenticated local channel between the Python SDK/CLI and the agent.
EMS	Lightrider's cloud platform (light-rider-platform) for fleet telemetry and cloud entropy.
M-23-02	The U.S. federal memorandum requiring agencies to inventory quantum-vulnerable cryptography.

2 Scope and Requirements

Scope is fixed by the PQC SDK guidance documents and the qSearch architecture notes. The single V1 scope change approved by Lightrider is the cloud-entropy mix-in (Section 6).

2.1 In scope (V1)

Area	V1 capability
qSearch discovery	Source-code and dependency scanning; certificate and protocol discovery; passive network/PCAP analysis; identification of RSA, DH, ECDH, ECDSA and other quantum-vulnerable cryptography
CBOM	Automatic CBOM export in JSON, CSV, and CycloneDX; M-23-02 inventory fields; executive summary; mapping to the Light Rider CBOM workbook
Entropy	wolfEntropy ESV-certified software entropy source, SP 800-90B health tests, fail-closed seeding/reseeding of the wolfCrypt DRBG
PQC services	ML-KEM key generation / encapsulation / decapsulation; ML-DSA signing / verification; opaque key handles only
TLS	Hybrid TLS 1.3 (classical + ML-KEM group) sample client and server, policy-profile driven (LIGHTRIDER_PQC_TRANSITION)
FIPS	Startup + conditional self-tests, approved-mode status, per-item validation-status reporting (module cert, ESV cert, PQC status)
EMS (optional)	Telemetry/CBOM synchronization; user-controllable cloud-entropy mix-in; disabled by default with a zero-network-traffic guarantee
Platforms	Windows 11 x86-64 (MSI + Windows service) and Linux x86-64 (deb/rpm + systemd unit)

2.2 Out of scope (V1)

Quantum Light hardware and QPU-derived entropy as *seed* sources; D-QKD; mobile; tactical hardware; customer-specific FIPS operational environments; multi-source entropy pooling; proprietary algorithms; automatic remediation/migration orchestration in qSearch; macOS. *Do not expand V1 scope without approval.*

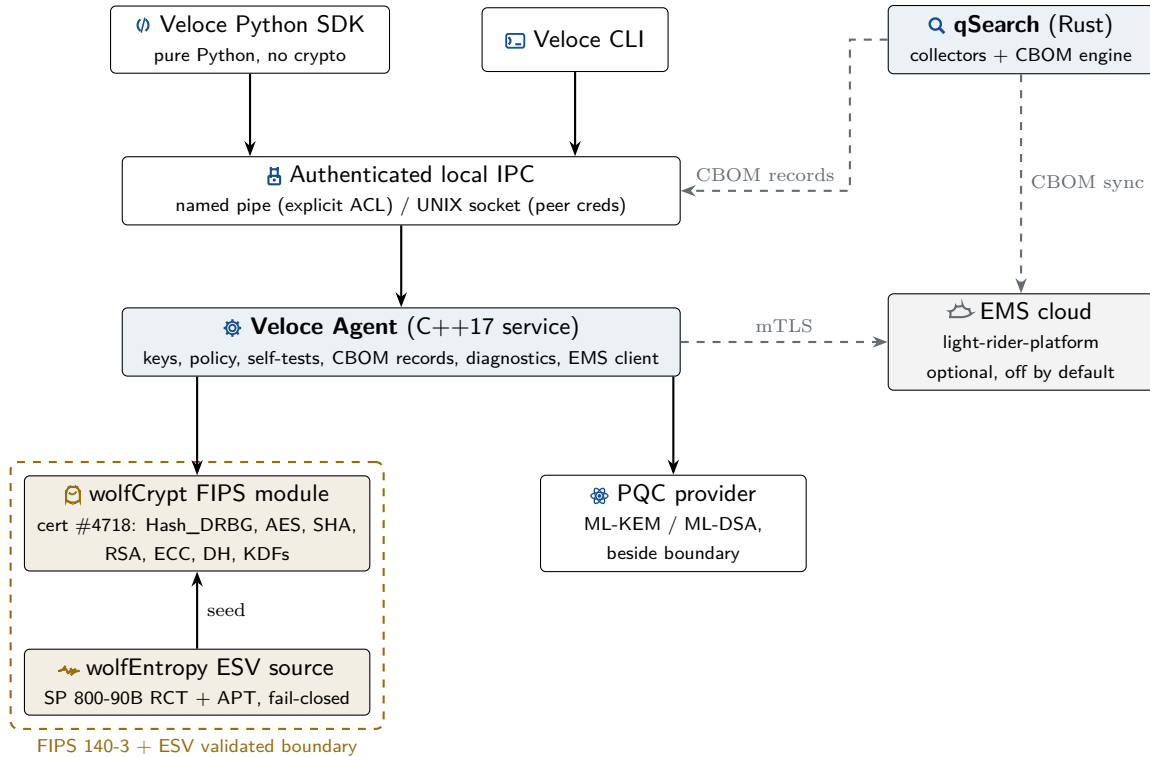
2.3 Standards and policy baseline

Standard	Role	Veloce usage
FIPS 140-3	Module validation	wolfCrypt module v5.2.1, CMVP cert #4718
SP 800-90A	DRBG	Hash_DRBG (SHA-256), 256-bit strength
SP 800-90B / ESV	Entropy source	wolfEntropy: RCT + APT health tests, ESV certificate
FIPS 203 / 204	PQC algorithms	ML-KEM-768 (default), ML-DSA
TLS 1.3 (RFC 8446)	Protocol	Hybrid groups, e.g. X25519MLKEM768
CycloneDX CBOM	Inventory format	qSearch + SDK CBOM export
OMB M-23-02	Federal policy	Inventory fields; scanning-first CBOM process
CISA ACIDI strategy	Discovery baseline	Coverage model for qSearch collectors

[OPEN: Federal “CBOM minimum elements” are pending; the CBOM schema module must be versioned so it can adopt them without breaking consumers.]

3 System Architecture

One native agent owns all keys, entropy, and FIPS state. Python and the CLI are thin, crypto-free IPC clients. qSearch is a separate Rust binary outside the FIPS boundary that shares only the CBOM data model with the SDK. EMS is reachable only from the agent, only when enabled.



3.1 Component responsibilities

Component	Language	Owns
Veloce Python SDK	Python	Public API, IPC client, typed results; no direct wolfCrypt loading; all cryptography via the agent.
Veloce CLI	Rust	Operator commands: status, self-test, CBOM export, diagnostics, scan.
Veloce Agent	C++17	wolfCrypt lifecycle, entropy, keys (opaque handles), policy profiles, FIPS self-tests, validation status, CBOM records, EMS client, diagnostics.
wolfCrypt module	C	FIPS 140-3 approved algorithms + Hash_DRBG (dynamic library; never statically linked; protected files never modified).
wolfEntropy	C	ESV-certified noise source + conditioning + health tests.
PQC provider	C	ML-KEM / ML-DSA from the wolfSSL tree, compiled beside the FIPS boundary (see 5.3).
qSearch	Rust	Collector adapters, normalization, correlation, classification, CBOM generation, risk scoring, reporting. Fully offline-capable.
EMS	cloud	Fleet telemetry, CBOM aggregation, policy distribution, cloud-entropy service.

Repository layout: `agent/`, `python/`, `cli/`, `qsearch/`, `ipc/` (versioned, length-prefixed message definitions), `installer/` (MSI + deb/rpm), `tests/` (unit, integration, interoperability, security), `examples/` (tls-client, tls-server), `docs/`, `cbom/`, `vendor/wolfssl/` (licensed source; restricted private repository, excluded from any public repo).

4 Pillar A: qSearch, Cryptographic Search

qSearch discovers cryptography, classifies quantum vulnerability, and maintains the CBOM. Collectors are replaceable adapters; the normalization, correlation, classification, CBOM, risk, and reporting layers are Lightrider-developed. qSearch runs entirely offline, outside the FIPS boundary.

4.1 Collector adapters (replaceable)

Adapter	Backend and method	Platform
Source-code scan	Implemented: recursive Rust walker, case-sensitive literal patterns, selected extensions, 4 MiB file limit; skips common build/vendor directories	Linux; portable source mode planned for Win/macOS
Dependency scan	Planned: package/lockfile + SBOM (SCA) analysis; links findings to packages, container images, and digests	Win + Linux + macOS
Certificate discovery	Implemented: PEM blocks, SHA-256 fingerprints and selected key OIDs; Linux certificate-store collector. Native Windows/macOS stores planned	Linux
Network / PCAP	Planned: Zeek (or Suricata) for live traffic and PCAP: TLS versions, cipher suites, key-exchange groups	Linux native
Windows capture	TShark/Wireshark + Npcap producing PCAP; analysis by Zeek on a Linux sensor, WSL, or analysis server (Zeek on Windows is experimental)	Windows
Endpoint / config	Implemented on Linux: <code>ldconfig</code> , library paths, <code>/proc/crypto</code> , <code>/proc/*/maps</code> , SSH, certificate stores, and policy files; Windows/macOS collectors planned	Linux

License policy: collectors must be Apache-2.0, MIT, or BSD. **[OPEN: GPL-licensed components (verify [CryptoScan](#), [Zeek](#), [Suricata](#) licenses) require legal approval of the distribution model before bundling; fallback is “bring-your-own-collector” integration.]**

4.2 Lightrider-owned layers

Normalization → correlation → quantum-vulnerability classification → CBOM generation → risk prioritization → reporting → EMS synchronization. These layers are written in Rust and are collector-agnostic.

4.3 One internal finding format

All collectors emit a single normalized record; downstream layers are tool-agnostic.

Core fields	Asset; application/process; algorithm; cryptographic service (key establishment, encrypted connection, digital signature, authentication, code/firmware signing); protocol; key/parameter length; certificate fingerprint; library/package; version; source of evidence; confidence level; last-observed date
Provenance label	directly observed statically detected vendor reported inferred manually validated
Workbook mapping	Provenance + method map onto the CBOM workbook <i>Discovery Findings</i> sheet (Discovery Method, Finding Confidence, Validation Status columns); see Appendix C.

4.4 Detection targets and outputs

V1 prioritizes *active* cryptography: key establishment, encrypted connections, digital signatures, authentication, code/firmware signing. Classification flags RSA, DH, ECDH, ECDSA (and small-key symmetric use) as

quantum-vulnerable; ML-KEM/ML-DSA usage is recognized and reported as PQC-ready.

Outputs: **JSON** (canonical), **CSV**, **CycloneDX CBOM**, **M-23-02 inventory fields**, **executive summary**, and **SDK runtime validation records** (the Veloce agent's own CBOM self-report merges into the same store).

4.5 Coverage assurance and outlier names

No finite keyword list can guarantee discovery of every crypto wrapper, alias, generated binding, renamed module, or proprietary API. qSearch therefore treats keyword matches as evidence, not proof, and uses a layered coverage model:

1. Curated rules include canonical standards, historical names (for example Kyber and Dilithium), vendor APIs, language bindings, protocol groups, OIDs, imports, package names, and configuration keys. Rules move to a versioned external JSON/TOML file with rule identifier, aliases, classification, risk, source, and regression fixture.
2. Structural collectors examine certificates, dependency manifests, linked/loaded libraries, symbols, processes, and protocol observations. These catch modules whose application-facing name does not contain a known algorithm keyword.
3. Every rule has positive, negative, alias, comment/test, and boundary fixtures. Controlled corpora measure precision, recall, false positives, and explicit blind spots. Unknown providers are reported as context for analyst review, never silently classified as safe.
4. Customer/vendor dictionaries may add organization-specific wrapper and product names without rebuilding qSearch. Signed rule-pack versions are recorded in every run log and CBOM record.

4.6 Optional AI triage

AI is an enrichment stage after deterministic collection; it is never the sole source of a CBOM fact. qSearch batches findings and, only with explicit tenant opt-in, submits the minimum necessary context (rule ID, path token, matched line and a bounded surrounding window) to a Lightrider AI gateway. The gateway may classify active code versus comments, tests, examples, or wrappers; suggest aliases; explain risk; and propose remediation. The original match, file/line evidence, model/version, prompt-policy version, confidence, and human disposition remain in the audit record.

The model-provider credential is stored once in Lightrider's server-side secret manager and is never shipped in qSearch, a wheel, a CLI flag, a repository, or a customer report. Customers authenticate to Lightrider with a scoped tenant credential; the gateway attaches the provider credential and applies quotas, redaction, retention, and audit policy. Calls are batched per scan/chunk, not made once per keyword. A bring-your-own-key mode and a fully local/offline model are optional deployment profiles. AI failure, denial, or timeout must not block deterministic scanning or alter the zero-network guarantee when EMS/AI is disabled.

4.7 Performance approach

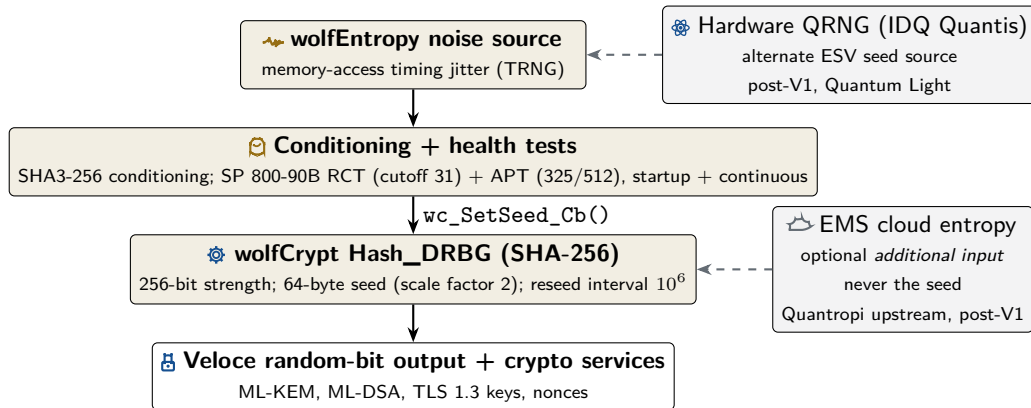
Current implementation: a single-threaded depth-first `read_dir` walk loads each readable file up to 4 MiB as UTF-8, then applies every literal pattern to every line. It does not use an index, AST, memory mapping, Aho–Corasick, or incremental hashes.

Target upgrade: parallel ignore-aware walking; one compiled Aho–Corasick/regex-set pass; bounded streaming; and incremental re-scan keyed on file hashes. AI runs only on the much smaller finding set. Benchmarks and false-positive/blind-spot reporting are release gates (Section 9).

5 Pillar B: Entropy Generator and FIPS Crypto Core

The crypto core consists of an ESV-certified entropy source feeding a validated DRBG, with PQC and TLS services above it. All failure modes are fail-closed.

5.1 Entropy pipeline



Wiring. The module itself makes no entropy claim. After loading the module and checking `wolfCrypt_GetStatus_fips()`, the implemented agent runs the `wolfEntropy` on-demand health test, registers `wc_GenerateSeed` with `wc_SetSeed_Cb()`, and instantiates the FIPS DRBG with `wc_InitRng()`. The build uses `--enable-wolfEntropy=nofallback`, so initialization or generation fails closed rather than silently falling back to `/dev/urandom`. The agent exposes `FipsCore::randomBytes()` as a callback to the separate PQC provider: ML-KEM keygen requests 64 bytes; ML-KEM encapsulation, ML-DSA keygen, and hedged ML-DSA signing request 32 bytes each. Applications never supply or retrieve raw entropy.

Fail-closed policy. If entropy initialization, a health test (RCT/APT), or a reseed fails, the agent transitions to a degraded state: all key generation, signing, encapsulation, and TLS handshakes are refused; `health()` and the validation-status API report the failure; an event-log entry and (if enabled) an EMS alert are emitted. There is no API that returns raw entropy, and entropy is never sent to EMS.

ESV evidence. `wc_Entropy_GetRawEntropy()` provides raw noise samples for SP 800-90B assessment. **[VENDOR: Confirm with wolfSSL the ESV certificate number and its applicability to our exact Windows 11 x86-64 and Linux x86-64 operating environments; entropy validations are OE-bound.]**

Entropy providers. V1 ships exactly one credited entropy provider: the `wolfEntropy` software source (ESV-verified). No hardware entropy is integrated in V1. Two quantum-entropy integrations follow post-V1: the **Quantropi** quantum-entropy service (QiSpace SEQR), delivered over the network through the EMS cloud-entropy channel as DRBG additional input (Section 6.2); and the IDQ Quantis QRNG as a local alternate ESV seed source for Quantum Light deployments. `list_entropy_providers()` enumerates providers, so new sources are added without API change.

5.2 FIPS module integration

- Module: `wolfCrypt FIPS 140-3 v5.2.1`, CMVP cert #4718; library `wolfSSL 5.9.2`. Built with `--enable-fips=v5` via autotools (CMake is not FIPS-capable in this tree). Loaded as a shared library / DLL, never statically linked; protected module files are never renamed, patched, or recompiled without `wolfSSL` approval.
- In-core integrity: HMAC-SHA256 check; build flow is `configure → make → ./fips-hash.sh → make → testwolfcrypt` (Appendix A).
- Self-tests: power-on self-test at load plus conditional algorithm self-tests (CASTs) for AES-CBC/GCM, HMAC-SHA-1/2/3, DRBG, RSA sign, ECDSA, ECC-CDH, DH-Z, and the TLS 1.2/1.3/SSH KDFs. The SDK's `run_fips_self_tests()` calls `wc_RunAllCast_fips()`, and status is read from `wolfCrypt_GetStatus_fips()`.
- Record for every build: DLL/so hash, source version, build settings, compiler, FIPS certificate, ESV certificate, operating environment (feeds the CBOM).
- The licensed source package does include Windows FIPS build material: `IDE/WIN10/wolfssl-fips.sln`, `wolfssl-fips.vcxproj`, and `user_settings.h`. Its README says the supplied library is currently built

statically and documents special in-core integrity linker settings.

- As checked on 2026-08-10, wolfSSL lists **Windows 10 Enterprise on Intel Core i5-1345U** (with and without PAA) and **Windows 11 Pro on Intel Core i7-1260P @ 2.10 GHz** as *pending operational environments*. NIST's CAVP record A4308 already enumerates these algorithm-test OEs, but the current CMVP certificate #4718 security policy does not list them in its approved tested-OE table. CAVP algorithm testing and a pending OE must not be represented as current CMVP module/OE approval. The earlier i7-1255U statement referred to a separate future module and was incorrect for this claim.
- The CMVP entry carries the caveat “no assurance of the minimum strength of generated SSPs”; entropy assurance therefore depends on the separately assessed entropy source. **[VENDOR: Obtain wolfSSL confirmation when the two Windows OEs are added to the published #4718 security policy, plus the approved dynamic-library configuration and exact v5.2.1 user_settings.h. Until then Windows is a development target, not a Veloce validated deployment claim.]**

5.3 PQC provider outside the FIPS boundary

The #4718 boundary contains classical algorithms only, and this FIPS build strips the SHAKE/XOF primitives that ML-KEM and ML-DSA require, so **PQC cannot link inside the v5 FIPS build**. V1 therefore compiles the wolfSSL ML-KEM/ML-DSA sources (FIPS 203/204, with fault-hardening flags) as a separate *PQC provider* library beside the boundary, drawing all randomness from the FIPS DRBG via the `*WithRandom` APIs.

- Validation status reports each fact separately (no generic `FIPS=True`): module validated + cert number; entropy ESV validated + cert number; PQC algorithm and its validation status; `pqc_inside_fips_boundary: false`; OE; self-test results. Values come from the installed module, never hardcoded.
- **[VENDOR: Track wolfSSL's FIPS v7 “PQ-FS” submission (boundary wrappers for ML-KEM, ML-DSA, LMS, XMSS, SLH-DSA + SHA-512 DRBG). Migrating to v7 flips `pqc_inside_fips_boundary` to true with no API change; the provider interface is designed for that swap.]**
- CAVP/ACVTS testing of the PQC provider (known-answer + interface tests) is planned so the CBOM can cite algorithm-level evidence even before v7. **[OPEN: Choose ACVTS scope and lab.]**

5.4 Hybrid TLS 1.3

Sample client and server negotiate a wolfSSL-supported hybrid group. Candidates: `X25519MLKEM768` (default), `SecP256r1MLKEM768`, `SecP384r1MLKEM1024`. The public API accepts a *policy profile*, never raw algorithm strings:

```
session = veloce.tls.connect(host=..., port=443, policy="LIGHTRIDER_PQC_TRANSITION")
```

[VENDOR: Do not hardcode the production group until wolfSSL confirms the group supported by the licensed bundle, its validation status, the approved build configuration, and interoperability expectations.]

5.5 Performance plan

Performance is verified by measurement. Using `wolfcrypt/benchmark` plus Veloce-level harnesses, each release publishes: DRBG throughput (MB/s, single and multi-threaded via per-thread RNG instances); entropy source fill rate; ML-KEM-768 keygen/encap/decap ops/s; ML-DSA sign/verify ops/s; hybrid TLS 1.3 handshakes/s; IPC round-trip latency. The baseline is captured at stage S1 on the reference environments; release gate = published performance report with no regression >10% against baseline. **[OPEN: Absolute targets to be set from the S1 baseline.]**

6 Pillar C: EMS Cloud Integration

EMS (`light-rider-platform`) is strictly optional. Veloce is fully functional with EMS disabled; this default is enforced by a zero-network-traffic test.

6.1 Modes and configuration

```
ems: { mode: disabled }   or   ems: { mode: enabled, endpoint: https://..., entropy_mixin:
off|on }
```

EMS may receive	Agent health; SDK and module versions; CBOM records; policy identifiers; self-test status; entropy-provider status; non-sensitive operational telemetry
EMS must never receive	Private keys; shared secrets; <i>raw entropy</i> ; DRBG seeds or state; unencrypted customer data; TLS session secrets

Transport: mTLS (hybrid TLS 1.3 once EMS supports it), device enrollment issuing a per-device identity; the agent is the only component that opens the connection. **[OPEN: EMS endpoint contract (REST vs gRPC), enrollment flow, and payload schemas are designed at stage S3 jointly with the platform team.]**

6.2 User-controllable cloud entropy (V1 addition)

EMS may deliver entropy *to* devices; devices never send entropy to EMS.

Design.

1. The agent (when `entropy_mixin: on`) periodically fetches an authenticated, freshness-checked entropy packet from EMS over mTLS.
2. The packet is injected into the wolfCrypt DRBG as SP 800-90A *additional input* on reseed/generate: it is cryptographically mixed in but contributes *zero* to the credited entropy claim. The local wolfEntropy ESV source remains the sole seed. The FIPS and ESV certificates are unaffected.
3. User control: enable/disable per config or at runtime via the SDK (`set_entropy_mixin(...)`); state is visible in `health()`, `list_entropy_providers()`, validation status, and the CBOM (`cloud_entropy_mixin: on/off`).
4. Fail-safe: EMS unreachable, stale, or malformed packets $\Rightarrow$ mix-in is skipped and local operation continues untouched; a status flag records the last successful mix-in. Cloud entropy can never block or degrade local crypto.

Post-V1, the same delivery channel carries quantum-sourced entropy: the Quantropi quantum-entropy service (QISpace SEQR) as the EMS upstream source, and Quantum Light QPU-derived entropy.

7 APIs and Deliverables

7.1 Public Python API (17 functions, fixed by the guidance)

Group	Functions
Lifecycle	<code>initialize()</code> , <code>health()</code> , <code>version()</code>
Discovery	<code>list_policy_profiles()</code> , <code>list_crypto_providers()</code> , <code>list_entropy_providers()</code> , <code>validation_status()</code>
ML-KEM	<code>mlkem_generate_keypair()</code> , <code>mlkem_encapsulate()</code> , <code>mlkem_decapsulate()</code>
ML-DSA	<code>mldsa_generate_keypair()</code> , <code>mldsa_sign()</code> , <code>mldsa_verify()</code>
TLS / FIPS	<code>configure_hybrid_tls()</code> , <code>run_fips_self_tests()</code> , <code>approved_mode_status()</code>
Reporting	<code>export_cbom()</code> , <code>generate_diagnostic_bundle()</code>

Private keys never cross the IPC channel: key APIs return a public key plus an opaque `private_key_handle`. The implementation adds `mldsa_generate_keypair()` (required to exercise signing) and the housekeeping operation `release_key()` for agent-side zeroization. V1 also adds `set_entropy_mixin()` / mix-in status inside `list_entropy_providers()`.

7.2 Deliverables checklist

Python wheel (`veloce_pqc-<ver>-py3-none-any.whl`); native agent; wolfCrypt shared library/DLL; qSearch binary; Windows MSI (Authenticode-signed, service registration, named-pipe ACLs, EMS-disabled default config, upgrade/repair/uninstall) and Linux deb/rpm (systemd unit, socket permissions); CLI; API reference; architecture document; security guide; FIPS integration guide; entropy-provider guide; SBOM and CBOM; sample TLS client/server; test vectors; performance report; known-limitations statement; reseller evaluation package; **Lightrider EULA** and **third-party notices file** (wolfSSL attribution + collector licenses; see Section 8.1); branding assets (logo artwork + ASCII banner, Section 7.4).

7.3 Platform-separated development and release layout

All operating systems remain in one repository and share protocol, algorithm, report, and conformance-test definitions. OS-specific code is isolated behind transport, library loader, service, system-discovery, and packaging interfaces; platform conditionals must not spread through cryptographic policy code. The target source layout is:

```
platform/linux/  Unix socket/peer credentials, dlopen, systemd, Linux collectors
platform/windows/  named pipe/ACL, LoadLibraryExW, Windows service, Windows collectors
platform/macos/  local IPC/peer authorization, dylib, launchd, Keychain/macOS collectors
core/, ipc/, python/, qsearch/  shared behavior and schemas
installer/{linux,windows,macos}/  platform package inputs
```

The existing Linux implementation remains in `agent/`, `cli/`, and `qsearch/src/system.rs` until the adapters are extracted; empty or planned platform directories do not imply support. Releases are assembled into explicit targets, never a mixed archive:

```
release/<version>/linux/x86_64/  tar/deb/rpm, ELF binaries and .sos
release/<version>/windows/x86_64/  signed MSI, PE binaries and DLLs
release/<version>/macos/{x86_64,arm64}/  signed/notarized package and .dylibs
release/<version>/common/  protocol/docs plus only genuinely portable files
```

Each target carries its own manifest, SHA-256 checksums, SBOM/CBOM, tested OS/CPU, validation claims, known limitations, and gate report. The current `scripts/make_release.sh` produces Linux x86-64 only; Windows and macOS release folders must not be emitted until their gates pass.

7.4 Startup branding

Client-visible entry points display Lightrider branding at startup. The banner's status line is populated from `validation_status()` at runtime, never hardcoded.

```
-- \ / / _ | | / _ \ / _ | _ |
 \ v / | _ | | _ | ( ) | ( _ | _ |
  \ / | _ | _ _ \ _ / \ _ | _ |
Lightrider Inc -- Veloce PQC SDK v<version>
FIPS 140-3 #4718 | ESV entropy: OK | approved mode: on
```

Surface	Behavior
Veloce CLI	Prints the ASCII banner + status line on interactive startup. Suppressed when stdout is not a TTY, with <code>--quiet</code> or <code>--json</code> , or when <code>VELOCE_NO_BANNER=1</code> ; machine-readable output is never polluted.
Veloce Agent	Writes the banner and status line to its startup log entry: Windows Event Log source on Windows, journald/syslog on Linux.
Python SDK	No output on <code>import</code> (library convention); banner available explicitly via <code>veloce.banner()</code> .
Installers	MSI UI dialogs carry Lightrider logo artwork; deb/rpm post-install prints a one-line branded confirmation with version and FIPS certificate.

Reports CBOM exports, diagnostic bundles, and the qSearch executive summary carry a product-identity header (name, version, certificates).

Branding assets live in `assets/branding/` (logo SVG/ICO for installers and documentation; `banner.txt` as the single source for the ASCII art).

8 Security and Compliance

Control area	V1 implementation
Key custody	All private keys and sensitive state live in the agent; opaque handles outside; zeroization on release (tested).
IPC hardening	Windows: named pipe <code>\\.\pipe\LightRider.PQC.v1</code> with an explicit ACL (service identity, authorized local user, local admins; deny anonymous/remote/unauthenticated; never the default descriptor). Linux: UNIX domain socket, mode 0660, <code>SO_PEERCREC</code> checks. Versioned length-prefixed protocol; no raw entropy, DRBG state, keys, or plaintext diagnostics on the channel.
Library loading	Windows: absolute path under <code>Program Files</code> , <code>SetDefaultDllDirectories</code> + <code>LoadLibraryExW</code> restricted search; never in <code>System32</code> or global <code>PATH</code> . Linux: absolute <code>dlopen</code> path, no <code>LD_LIBRARY_PATH</code> trust; DLL/so hash verified against the recorded build hash.
Service identity	Windows service <code>VelocePqcAgent</code> under <code>NT AUTHORITY\LocalService</code> ; Linux systemd unit with a dedicated low-privilege user, <code>ProtectSystem=strict</code> . [VENDOR: Confirm service-account choice against wolfSSL operational-environment requirements.]
Certificates reported	FIPS 140-3 cert #4718 (module v5.2.1) + ESX entropy certificate, each bound to its exact OE; surfaced in validation status and CBOM.
Data hygiene	Diagnostic bundles are redaction-tested; CBOM output contains metadata only: never keys, seeds, passwords, tokens, or customer plaintext (mirrors the CBOM workbook rule).
Supply chain	Licensed wolfSSL source confined to a restricted private repo (<code>vendor/</code>); reproducible FIPS builds; signed installers; SBOM per release; qSearch collector licenses vetted (Apache-2.0/MIT/BSD).

8.1 Licensing and distribution model

Lightrider holds a wolfSSL commercial license (Agreement v.01-24 in the bundle); **Veloce ships under Lightrider’s own commercial license (EULA)**, wrapping wolfCrypt as an embedded component. The agreement’s terms impose the following requirements:

Agreement fact	What Veloce must do
§3: distribute the “Combined Program” in <i>object code</i> only	Ship wolfCrypt exclusively as the compiled DLL/.so inside Veloce installers: never source, never as a standalone library. Already matched by the dynamic-library design and the restricted <code>vendor/</code> repo.
§3: license is contingent on the Combined Program <i>not</i> being a cryptography library that competes with the Products	[VENDOR: Confirm with wolfSSL that Veloce (a PQC SDK exposing crypto services through the local agent, not an embeddable crypto library) is the named Program on the Order Form.] Supporting position: Veloce exposes cryptographic services through the local agent (opaque handles), not a linkable crypto library.
§3: per-Developer licensing (anyone who links, compiles, or edits code using the Products)	Count agent/core developers against the Order Form seat count; qSearch and Python SDK developers never touch wolfCrypt and do not count. Track seats in the repo access list.
§4: retain wolfSSL copyright notices; attribution in documentation permitted; no sublicense/transfer	Keep wolfSSL notices in the shipped binaries and a third-party notices file ; docs state Veloce is “built with wolfCrypt (FIPS 140-3 cert #4718)”. The Lightrider EULA licenses <i>Veloce</i> , and must not purport to sublicense wolfSSL itself.

§5: confidentiality (3 years / trade-secret duration)	FIPS source bundle and build configs stay in the restricted private repo; excluded from any public repository and from customer deliveries (including the Customer Operational-Environment Package).
§7: end-user rights survive termination	End-user license continuity is reflected in the Lightrider EULA.
§10: support covers updates to the licensed version, not upgrades	[VENDOR: The FIPS v7 “PQ-FS” migration is an upgrade; budget a new wolfSSL Order Form for it.]
§11: FAR 12.211/12.212, DFARS 252.227-7015 government rights	Mirror the restricted-rights language in the Lightrider EULA; required for the M-23-02 federal customer base.

Layered result: **Lightrider EULA** (Veloce SDK, all Lightrider-owned code) → **wolfSSL commercial license** (embedded wolfCrypt object code, invisible to the customer beyond notices) → **permissive licenses** (Apache-2.0/MIT/BSD qSearch collectors, listed in the notices file). qSearch and the Python SDK contain no wolfSSL code, so Lightrider may license them freely under its own terms.

9 Test Gates and Acceptance

Release is gate-based: V1 does not ship until every gate passes on both platforms.

Gate group	Must pass
FIPS & entropy	wolfCrypt test suite; startup + conditional self-tests; wolfEntropy init and health tests; induced entropy-failure and reseed-failure tests (fail-closed verified); DRBG continuous-test behavior.
PQC	ML-KEM keygen/encap/decap and ML-DSA sign/verify known-answer tests; fault-harden builds; negative tests (corrupted ciphertext/signature).
TLS	Hybrid TLS 1.3 client/server interop; negotiated-group reporting; certificate validation; downgrade rejection; fragmented handshakes; unsupported-server behavior; EMS on/off operation.
IPC & platform	Named-pipe/socket authorization; unauthorized-local-user rejection; remote-access rejection; DLL substitution and search-path attacks; key zeroization; diagnostic redaction.
EMS	Zero network traffic when disabled; telemetry allow-list enforcement when enabled; entropy mix-in on/off behavior incl. EMS-unavailable fail-safe.
qSearch	Controlled sample environments containing known RSA, ECDH, ECDSA, DH, ML-KEM, ML-DSA implementations: expected detections found; false positives and blind spots explicitly reported; CBOM accuracy vs. ground truth; CycloneDX schema validation; offline-mode run.
Packaging & perf	MSI install/upgrade/repair/uninstall; deb/rpm equivalents; config preserved across upgrades; performance and memory benchmarks published.

V1 completion criteria (condensed): Python SDK talks only through the agent; wolfCrypt loaded dynamically; ESV source seeds the DRBG; hybrid TLS works client/server; full local operation with EMS disabled; EMS enable exposes no secrets; validation status accurate per-item; CBOM export automatic; installers produce securely configured services; all gates green.

10 Execution Guide: What To Do, In Order

Work proceeds as a dependency-ordered sequence of stages, each closed by an objective gate. The two pillars run in parallel after S0. Vendor items (**[VENDOR:]**) are on the critical path and are opened at S0.

Stage	Target	What to do / exit gate
-------	--------	------------------------

S0	Foundations	Repo + CI skeleton; Linux FIPS build brought up (<code>--enable-fips=v5</code> <code>--enable-wolfEntropy=nofallback</code> , fips-hash flow, <code>testwolfcrypt</code> green); IPC protocol v1 drafted; wolfSSL engagement opened (Windows DLL build configuration, ESV cert applicability, hybrid group, v7 timeline). <i>Gate G0: FIPS module passes self-tests on the Linux reference machine.</i>
S1	Entropy + core	Agent skeleton (Linux daemon + Windows service); wolfEntropy wired via <code>wc_SetSeed_Cb</code> ; fail-closed state machine; PQC provider built beside boundary + KATs; validation-status API; performance baseline. <i>Gate G1: entropy pipeline + ML-KEM/ML-DSA pass all KATs and failure-injection tests on both OSes.</i>
S2	qSearch	Rust engine + finding schema; source/dependency/ certificate collectors; Zeek/PCAP pipeline (Linux native, Windows capture); classification + CBOM (JSON/CSV/CycloneDX); sample-environment accuracy runs. <i>Gate G2: qSearch detects all planted crypto in the controlled environments and reports false positives/blind spots. (Parallel to S1.)</i>
S3	TLS + EMS	Hybrid TLS 1.3 samples + interop; Python SDK surface complete; CLI; EMS telemetry client + enrollment; cloud-entropy mix-in; zero-traffic verification. <i>Gate G3: TLS + EMS gate groups green.</i>
S4	Harden & ship	Full security test battery (IPC, DLL, zeroization, redaction); packaging (MSI, deb/rpm, wheel); docs set; performance report; known-limitations statement; reseller evaluation package. <i>Gate G4 = V1 release candidate.</i>

10.1 Component-to-repository traceability

Each repository directory implements a specification section and is accepted by an exit gate.

Repository path	Specification	Gate
vendor/wolfssl/	Appendix A build recipes; §8.1 license constraints	G0
ipc/	§3 IPC protocol; §8 channel ACLs	G1
agent/	§5 entropy pipeline, FIPS integration, PQC provider; §8 key custody, loading, service identity	G1
qsearch/	§4 collectors, finding format, outputs	G2
cbom/	§4 output formats; Appendix C field mapping	G2
python/	§7 API surface; §6 EMS controls	G3
cli/	§7 operator commands; §7.4 banner	G3
examples/	§5 hybrid TLS 1.3 client/server	G3
tests/	§9 gate definitions	G0–G4
installer/	§7 packaging; §8 secure defaults	G4
assets/branding/	§7.4 logo + banner assets	G4
docs/	§7 documentation deliverables	G4
docs/two-server-manual-test.md	Manual GitHub-synchronized two-host ML-DSA/ML-KEM interoperability procedure	G3

Post-V1 roadmap: wolfSSL FIPS v7 migration (PQC inside boundary); IDQ Quantis QRNG as alternate local ESV seed source (Quantum Light); Quantropi QiSpace SEQR entropy as the EMS upstream, delivered over the mix-in channel; QPU-derived cloud entropy; macOS; multi-source entropy pooling; qSearch remediation orchestration.

11 Risks and Open Items

Risk / item	Prob.	Impact	Mitigation
-------------	-------	--------	------------

Windows FIPS dynamic-library configuration + ESV OE applicability unconfirmed; the Windows 10/i5-1345U and Windows 11/i7-1260P environments are pending and not yet in the current #4718 security-policy tested-OE table	M	H	Engage wolfSSL at S0; treat CAVP records as algorithm evidence only; make no Windows validated-deployment claim until the CMVP OE update and exact build guidance are published.
PQC outside FIPS boundary until v7	certain	M	Honest per-item status reporting (already the documented design); ACVTS algorithm evidence; v7 migration path built into the provider interface.
PQC implementation defects (cf. CVE-2026-6330, ML-KEM ARM64 ciphertext-comparison flaw)	L-M	H	Stay current with wolfSSL releases; KAT + negative tests in CI; x86-64-only scope in V1 limits exposure.
Hybrid TLS production group unconfirmed	M	M	Policy-profile indirection means the group is config, not API; default X25519MLKEM768 pending confirmation.
wolfSSL §3 compete clause: Veloce could be read as a “cryptography library”	M	H	Paper Veloce as the named Program on the Order Form at S0; agent-service architecture (no linkable crypto API) is the supporting position; legal review of the Lightrider EULA wrap.
Collector licensing (GPL) blocks bundling	M	M	Legal review at S0; adapters keep collectors replaceable; bring-your-own-collector fallback.
wolfEntropy throughput insufficient for high-rate workloads	M	M	Per-thread DRBG instances (reseeds amortized); measure at the S1 baseline; QRNG roadmap as high-rate alternative.
Federal CBOM minimum elements published mid-project	M	L	Versioned CBOM schema module; workbook already flags mandatory update on issuance.
Zeek-on-Windows immaturity	certain	L	Documented capture-and-forward design (TShark/Npcap → Linux/WSL sensor); native Windows analysis revisited post-V1.

A Build Recipes

Linux FIPS module (bundle on hand)

```
cd vendor/wolfssl
./configure --enable-fips=v5 --enable-wolfEntropy=nofallback
make
./fips-hash.sh  (updates the in-core HMAC-SHA256 verify hash)
make
./wolfcrypt/test/testwolfcrypt  (must report all tests passed)
```

Notes: autotools is the only FIPS-capable build path in this tree; `--enable-fips` also turns on reproducible builds. Record DLL/so SHA-256, source version, compiler, and flags into the CBOM store at build time.

PQC provider (beside boundary)

Separate non-FIPS build of the same wolfSSL tree (or standalone compilation of `wc_mlkem.*` and `wc_mldsa.*` with SHAKE enabled), configured with `--enable-mlkem --enable-mldsa` plus the fault-hardening flags `WC_MLKEM_FAULT_HARDEN` and `WC_MLDSA_FAULT_HARDEN`; randomness supplied exclusively from the FIPS DRBG via the `*WithRandom` APIs; distinct library name and symbol namespace so the two builds can never be confused at load time.

Windows

Use the wolfSSL-provided IDE/WIN10 FIPS solution (`wolfssl-fips.sln`), Platform `x64`, per-wolfSSL-approved configuration only. Local inspection confirms the solution, project, and `user_settings.h` exist. The included README currently says to build a static library and documents integrity-sensitive linker settings; Veloce's dynamic-loading requirement therefore remains unresolved. wolfSSL lists Windows 10 Enterprise/i5-1345U (with and without PAA) and Windows 11 Pro/i7-1260P as pending OEs; the A4308 CAVP record is not by itself CMVP module/OE approval. **[VENDOR: Obtain the approved DLL configuration, exact user guide/settings, and published #4718 OE update before enabling the Windows FIPS claim.]** Then implement the named-pipe transport, restricted DLL loading, Windows service, Windows collectors, MSI, and the same hash-verify-record flow behind platform adapters.

B Manual Two-Host Interoperability Test

The executable step-by-step procedure is maintained in `docs/two-server-manual-test.md`. It assigns `168.231.75.64 (srv811871.hstgr.cloud)` as signer/ML-KEM sender and `93.127.215.63 (srv862034.hstgr.cloud)` as verifier/ML-KEM recipient. Both hosts clone the same commit from the private GitHub repository using independent read-only deploy keys; the licensed wolfSSL bundle is provisioned separately and must never enter GitHub. Each host builds, gates, and runs its own local agent. Public keys, ML-DSA signature bundles, and ML-KEM ciphertext move through SSH/SCP; private handles and shared secrets never leave their owning host. The test compares shared-secret SHA-256 fingerprints and records commit, OS/CPU, validation, and gate evidence. It does not claim to implement authenticated payload encryption or hybrid TLS.

C CBOM Field Mapping (SDK → workbook)

The Veloce agent's self-report and qSearch findings both flow into the Light Rider CBOM workbook schema:

Veloce report field	CBOM Inventory column
Crypto library / module + version	Crypto Library/Provider; Library Version; Cryptographic Module
FIPS certificate (#4718)	FIPS 140-3 Certificate
Entropy provider + ESV certificate	Entropy Source; ESV Status/Certificate
Algorithm + parameter set (e.g. ML-KEM-768)	Algorithm; Key/Parameter Size; Target PQC Algorithm
TLS version + hybrid group	Protocol/Version; Cipher Suite
Approved-mode + self-test status	Notes / runtime validation record
OS edition, build, CPU arch	Operating System (Supplier sheet); Environment
qSearch finding provenance + confidence	Discovery Findings: Discovery Method, Finding Confidence, Validation Status, False Positive?, Blind Spot/Limitation
Scan runs	Scanning Log sheet (tool/method, findings counts, false positives)

Additional SDK-only CBOM fields (JSON/CSV export): agent version, wolfCrypt DLL/so name + SHA-256, EMS enabled/disabled, cloud-entropy mix-in state, installation and last-verified timestamps.

D References

- OMB Memorandum M-23-02, *Migrating to Post-Quantum Cryptography* (Nov 2022).
- CISA, *Strategy for Migrating to Automated Post-Quantum Cryptography Discovery and Inventory Tools* (Sep 2024).
- CISA, *2026 Minimum Elements for a Software Bill of Materials* (Jul 2026).
- NIST FIPS 203 (ML-KEM), FIPS 204 (ML-DSA); FIPS 140-3; SP 800-90A/B.
- NIST CMVP validated-modules search (wolfSSL):
<https://csrc.nist.gov/projects/cryptographic-module-validation-program>.
- NIST CMVP certificate #4718 and security policy (checked 2026-08-10):
<https://csrc.nist.gov/projects/cryptographic-module-validation-program/certificate/4718>.

7. NIST CAVP wolfCrypt product/A4308 records, including the Windows algorithm-test environments; CAVP evidence is not CMVP OE approval:
<https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/details?product=16876>.
8. wolfSSL FIPS status page, “Pending Operating Environments currently in testing” (checked 2026-08-10): <https://www.wolfssl.com/license/fips/>.
9. CryptoScan: <https://github.com/csnip/cryptoscan>; Zeek packet analysis: <https://docs.zeek.org>.
10. wolfSSL 5.9.2 release (Jun 23, 2026), commercial FIPS bundle `linuxv5.2.1`; wolfSSL FIPS-FAQ v2.3.
11. Minimus, *From SBOM to CBOM* and *How to Use CBOMs in Containerized Environments* (industry perspective).
12. Light Rider CBOM Starter Template (`Light_Rider_CBOM_Template_Updated.xlsx`).